

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory Work No. 1

Cryptography

Caesar Cipher

Standard and two-key versions

over the Romanian alphabet

Supervisors:

Poștaru Andrei, *University lecturer*

Maia Zaica, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Short theory **3**

1.1 Objectives and the Romanian alphabet 3

1.2 Task 1.1: the standard Caesar cipher 3

1.3 Task 1.2: Caesar cipher with two keys 4

2 Implementation **5**

2.1 Program structure and letter encoding 5

2.2 Normalization and message validation 6

2.3 Building and displaying the permuted alphabet 7

2.4 Encryption and decryption 8

2.5 Numeric keys, retry loops and the console menu 9

3 Results **10**

3.1 Task 1.1: encryption and decryption 10

3.1.1 Recovering the standard-cipher plaintext 11

3.2 Task 1.2: encryption and decryption 12

3.2.1 Recovering the two-key plaintext 13

3.3 Invalid numeric keys and message characters 14

3.4 Invalid keywords 15

3.5 Romanian letters and boundary checks 16

4 Conclusions **17**

4.1 Source code 17

Bibliography **18**

1 Short theory

1.1 Objectives and the Romanian alphabet

The aim of Laboratory Work No. 1 is to implement the standard Caesar cipher and a Caesar cipher with a keyword-based alphabet permutation, to understand modular encryption and decryption, and to compare their security. Both tasks use the 31-letter Romanian alphabet and the encoding specified in the laboratory assignment [1].

Each letter is represented by its position, starting at zero. Table 1.1 shows the complete encoding. Character codes such as ASCII or Unicode values are not used to calculate the shift.

Table 1.1 - Romanian alphabet and the required numeric encoding

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	
Code	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	

1.2 Task 1.1: the standard Caesar cipher

A Caesar cipher replaces every plaintext letter with the letter located a fixed number of positions further along the alphabet. The secret key is an integer $k \in \{1,2,\ldots,30\}$. For a plaintext index x and a ciphertext index y, encryption and decryption are

$$y = E_k(x) = (x+k) \bmod 31, \tag{1.2.1}$$

$$x = D_k(y) = (y-k+31) \bmod 31. \tag{1.2.2}$$

Adding 31 during decryption prevents a negative remainder. This is sufficient because the accepted key is at most 30. For example, with $k = 3$, the letter C has index 4 and becomes F, at index 7; Z has index 30 and becomes Â, at index 2. Decrypting Â gives $(2-3+31) \bmod 31 = 30$, which is Z.

The standard cipher has exactly 30 permitted keys; $k = 0$ is excluded because it would leave the text unchanged. An attacker can decrypt the ciphertext with each of the 30 keys and inspect the candidates. This small key space makes exhaustive search practical.

1.3 Task 1.2: Caesar cipher with two keys

The second version uses a numeric key $k_1 \in \{1, \ldots, 30\}$ and a keyword k_2 containing at least seven Romanian letters. The keyword is converted to uppercase. Its letters are placed first, retaining only the first occurrence of each letter, and the unused letters of the Romanian alphabet are appended in their original order. The resulting alphabet P contains all 31 letters exactly once.

If $\text{pos}_P(a)$ denotes the position of letter a in P, the letter-level formulas are

$$E_{k_1,P}(a) = P[(\text{pos}_P(a) + k_1) \bmod 31], \tag{1.3.1}$$

$$D_{k_1,P}(b) = P[(\text{pos}_P(b) - k_1 + 31) \bmod 31]. \tag{1.3.2}$$

Both the lookup of the input letter and the lookup of the output letter use the same permuted alphabet. The keyword changes the alphabet order, while the numeric key determines the cyclic shift.

For $k_2 =$ `CRIPTOGRAFIE`, the distinct keyword letters are `CRIPTOGAFE`. Appending the remaining letters gives

`CRIPTOGAFEĂÂÂBDHÎJKLMNQSȘȚUVWXYZ`

With $k_1 = 3$, C at position 0 becomes P at position 3, while I at position 2 becomes O at position 5.

Considering every possible alphabet permutation gives the theoretical number of parameter pairs

$$|K_{\text{pairs}}| = 30 \cdot 31! \approx 2.47 \times 10^{35}.$$

This counts pairs of a shift and an alphabet ordering, rather than keyword strings: repeated letters can make different keywords produce the same ordering. Different parameter pairs can also produce the same substitution, so the number of distinct encryptions is at most 31!. Restricting keywords to ordinary words reduces the practical search space further.

Despite this larger theoretical key space, the cipher remains monoalphabetic: a given plaintext letter always maps to the same ciphertext letter. Letter counts and repeated-letter patterns are preserved under renaming, so frequency analysis can still be used. Neither version provides security suitable for protecting modern confidential information.

2 Implementation

2.1 Program structure and letter encoding

The implementation is a C# console program in `lab_01/code/Program.cs`. `RomanianAlphabet` provides shared normalization, validation, letter conversion and modular shifting. `CaesarCipher` implements Task 1.1, `TwoKeyCaesarCipher` implements Task 1.2, and `CipherTablePrinter` displays the numeric and letter mappings. `Program` manages the menu and input retry loops. The following excerpts are read directly from the existing source file; empty lines are compressed for readability.

```
    public const string Letters = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ";
    public const int Size = 31;
```

Comment. The alphabet string exactly matches Table 1.1, and its size is explicitly set to 31.

```
        for (int i = 0; i < text.Length; i++)
        {
            int index = alphabet.IndexOf(text[i]);
            if (index < 0)
            {
                throw new ArgumentException($"Character '{text[i]}' is not in the alphabet.");
            }
            numbers[i] = index;
        }
```

Comment. `IndexOf` converts each letter to its position in the supplied alphabet. For Task 1.1 this is the original alphabet; for Task 1.2 it is the permuted alphabet. A missing letter causes an exception, rather than an invalid numeric index.

```
    public static string ToText(int[] numbers, string alphabet)
    {
        char[] result = new char[numbers.Length];
        for (int i = 0; i < numbers.Length; i++)
        {
            result[i] = alphabet[numbers[i]];
        }
        return new string(result);
    }
    public static int[] ShiftNumbers(int[] numbers, int key)
    {
        int[] shifted = new int[numbers.Length];
        for (int i = 0; i < numbers.Length; i++)
        {
            shifted[i] = (numbers[i] + key + Size) % Size;
        }
        return shifted;
    }
```

Comment. `ToText` maps the shifted indices back to letters. `ShiftNumbers` implements modular arithmetic. Encryption passes a positive key; decryption passes its negative. Adding `Size` keeps the remainder nonnegative for the validated key range.

2.2 Normalization and message validation

```
    public static string Normalize(string text)
    {
        ArgumentNullException.ThrowIfNull(text);
        return text.Replace('ş', 'ș')
            .Replace('Ş', 'Ș')
            .Replace('ţ', 'ț')
            .Replace('Ţ', 'Ț')
            .ToUpperInvariant();
    }
```

Comment. Normalization replaces the cedilla variants ş/Ş and ţ/Ţ with the Romanian comma-below letters ș/Ș and ț/Ț, then converts the string to uppercase. This supports both uppercase and lowercase input from different keyboards.

```
    public static void Validate(string text, bool allowSpaces = true)
    {
        foreach (char letter in text)
        {
            if (allowSpaces && letter == ' ')
                continue;
            if (!Letters.Contains(letter))
            {
                throw new ArgumentException(
                    $"Character '{letter}' is not allowed. "
                        + $"Allowed letters: {Letters}"
                        + (allowSpaces ? " and spaces." : ".")
                );
            }
        }
    }
```

Comment. Validation accepts only letters from the explicit alphabet. Ordinary spaces are allowed in messages. An error identifies the rejected character and lists the allowed letters. Digits, punctuation, tabs and other characters are rejected.

The helper `Prepare` normalizes and validates the input before removing all ordinary spaces. Both encryption and decryption use this preparation. Consequently, decryption recovers uppercase text without spaces; the original capitalization and word spacing cannot be reconstructed.

```
    public static string PrepareKeyword(string keyword)
    {
        keyword = Normalize(keyword);
        Validate(keyword, allowSpaces: false);
        if (keyword.Length < 7)
        {
            throw new ArgumentException("Keyword must contain at least 7 letters.");
        }
        return keyword;
    }
```

Comment. Keywords are normalized in the same way as messages, but spaces are forbidden and the normalized keyword must contain at least seven letters. This length check occurs before duplicate removal, so the keyword does not need seven *distinct* letters.

2.3 Building and displaying the permuted alphabet

```
    public string BuildPermutedAlphabet(string keyword)
    {
        keyword = RomanianAlphabet.PrepareKeyword(keyword);
        string result = "";
        // Add unique letters from the keyword
        foreach (char letter in keyword)
        {
            if (!result.Contains(letter))
            {
                result += letter;
            }
        }
        // Append remaining Romanian alphabet letters
        foreach (char letter in RomanianAlphabet.Letters)
        {
            if (!result.Contains(letter))
            {
                result += letter;
            }
        }
        return result;
    }
```

Comment. The first loop keeps the first occurrence of each keyword letter. The second loop appends every unused Romanian letter in the prescribed order. As a result, the alphabet remains a permutation of all 31 letters even when the keyword contains duplicates or diacritics.

```
    public void PrintEncryptionTable(int key, string keyword)
    {
        RomanianAlphabet.ValidateKey(key);
        string alphabet = BuildPermutedAlphabet(keyword);
        Console.WriteLine($"\nKeyword: {RomanianAlphabet.Normalize(keyword)}");
        Console.WriteLine($"Original alphabet: {RomanianAlphabet.Letters}");
        Console.WriteLine($"Permuted alphabet: {alphabet}");
        CipherTablePrinter.Print(alphabet, key);
    }
```

Comment. Task 1.2 displays the normalized keyword, the original alphabet and the permuted alphabet, making the ordering available for manual verification. Both cipher classes also print an encryption table with four rows: `x`, `m`, `x+k` and `c`. The table always shows the forward mapping, including when the selected operation is decryption; the inverse operation itself uses the negative shift.

2.4 Encryption and decryption

```
    public string Encrypt(string message, int key)
    {
        RomanianAlphabet.ValidateKey(key);
        message = RomanianAlphabet.Prepare(message);
        int[] numbers = RomanianAlphabet.ToNumbers(message, RomanianAlphabet.Letters);
        int[] encrypted = RomanianAlphabet.ShiftNumbers(numbers, key);
        return RomanianAlphabet.ToText(encrypted, RomanianAlphabet.Letters);
    }
    public string Decrypt(string message, int key)
    {
        RomanianAlphabet.ValidateKey(key);
        message = RomanianAlphabet.Prepare(message);
        int[] numbers = RomanianAlphabet.ToNumbers(message, RomanianAlphabet.Letters);
        int[] decrypted = RomanianAlphabet.ShiftNumbers(numbers, -key);
        return RomanianAlphabet.ToText(decrypted, RomanianAlphabet.Letters);
    }
```

Comment. The standard cipher first validates the numeric key and prepares the message. It then converts letters to positions in the original alphabet, applies $+k$ for encryption or $-k$ for decryption, and converts the resulting positions back to letters.

```
    public string Encrypt(string message, int key, string keyword)
    {
        RomanianAlphabet.ValidateKey(key);
        message = RomanianAlphabet.Prepare(message);
        string alphabet = BuildPermutedAlphabet(keyword);
        int[] numbers = RomanianAlphabet.ToNumbers(message, alphabet);
        int[] encrypted = RomanianAlphabet.ShiftNumbers(numbers, key);
        return RomanianAlphabet.ToText(encrypted, alphabet);
    }
    public string Decrypt(string message, int key, string keyword)
    {
        RomanianAlphabet.ValidateKey(key);
        message = RomanianAlphabet.Prepare(message);
        string alphabet = BuildPermutedAlphabet(keyword);
        int[] numbers = RomanianAlphabet.ToNumbers(message, alphabet);
        int[] decrypted = RomanianAlphabet.ShiftNumbers(numbers, -key);
        return RomanianAlphabet.ToText(decrypted, alphabet);
    }
```

Comment. The two-key cipher follows the same steps, but builds the keyword alphabet first. Both `ToNumbers` and `ToText` receive that alphabet. Decryption must use the same numeric key and the same keyword-derived ordering as encryption.

2.5 Numeric keys, retry loops and the console menu

```
    public static void ValidateKey(int key)
    {
        if (key < 1 || key > 30)
        {
            throw new ArgumentException("Numeric key must be an integer between 1 and 30.");
        }
    }
```

Comment. The cipher methods independently enforce the numeric key interval $1 \leq k \leq 30$, so invalid keys are rejected even if a method is called outside the console menu.

```
    private static int ReadNumericKey()
    {
        while (true)
        {
            Console.Write("Enter numeric key (1-30): ");
            string? input = Console.ReadLine();
            if (int.TryParse(input, out int key) && key >= 1 && key <= 30)
            {
                return key;
            }
            Console.WriteLine("Invalid key. Enter an integer between 1 and 30.");
        }
    }
```

Comment. The console accepts a key only if `int.TryParse` succeeds and the value lies between 1 and 30. Otherwise it displays the permitted range and asks for the key again.

```
    private static string ReadKeyword()
    {
        while (true)
        {
            Console.Write("Enter keyword (at least 7 letters): ");
            string keyword = Console.ReadLine() ?? "";
            try
            {
                return RomanianAlphabet.PrepareKeyword(keyword);
            }
            catch (ArgumentException ex)
            {
                Console.WriteLine($"Invalid keyword: {ex.Message}");
            }
        }
    }
```

Comment. The keyword reader catches validation exceptions, displays the reason and repeats the prompt. `ReadMessage` uses the same pattern with `Prepare`, so invalid message characters also cause a retry. `ReadChoice` validates the available menu options.

The menu selects a cipher, then encryption or decryption. The user enters the numeric key, the keyword when required, and the message. The program prints the mapping table, prepared input and result, then returns to the menu; option 0 exits. The substitution uses explicit alphabet indices.

3 Results

3.1 Task 1.1: encryption and decryption

The unchanged program was compiled with the .NET 10 SDK. The screenshots show a live local console viewer connected to the program's standard input and output. The viewer only displays the console session; all cipher logic and validation come from `Program.cs`.

For the input `cifrul cezar` and $k = 3$, the prepared plaintext is `CIFRULCEZAR`. Encryption produces `FKITXOFHÂBT`; decrypting this ciphertext with the same key returns `CIFRULCEZAR`. These outputs use the Romanian alphabet, so they differ from the English-alphabet example in the assignment.



Figure 3.1 - Task 1.1: encryption with the numeric key 3

Manual check. The first three letters of `CIFRULCEZAR` are C, I and F, with positions 4, 10 and 7. Adding 3 gives positions 7, 13 and 10, or F, K and I. The Z at position 30 becomes Â at position 2.

3.1.1 Recovering the standard-cipher plaintext

The user selects the standard cipher and operation 2, then supplies key 3 and ciphertext `FKITXOFHÂBT`. The result in Figure 3.2 is `CIFRULCEZAR`, matching the prepared input from the encryption run.



Figure 3.2 - Task 1.1: decryption with the numeric key 3

Manual check. F at position 7 becomes C at position $(7-3+31) \bmod 31 = 4$. K at position 13 becomes I at position 10. Â at position 2 becomes Z at position $(2-3+31) \bmod 31 = 30$. Spaces and lowercase letters are absent because preparation removes them.

3.2 Task 1.2: encryption and decryption

For $k_1 = 3$ and the lowercase keyword `criptografie`, the program displays `CRIPTOGRAFIE` and the alphabet `CRIPTOGAFEĂÂBDHÎJKLMNQSȘȚUVWXYZ`. Encrypting `cifrul cezar` produces `POÂTXQPBIĂT`. Decryption with the same two keys restores `CIFRULCEZAR`.

The printed table also permits manual checking: C maps to P, I to O, F to Â, and Z wraps from position 30 to position 2, which contains I.

Romanian Caesar Cipher C# program · live standard input/output

```
=== Romanian Caesar Cipher ===
1. Standard Caesar Cipher
2. Two-Key Caesar Cipher
0. Exit
Choose cipher: 2

1. Encrypt
2. Decrypt
Choose operation: 1
Enter numeric key (1-30): 3
Enter keyword (at least 7 letters): criptografie
Enter message: cifrul cezar

Keyword: CRIPTOGRAFIE
Original alphabet: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ
Permuted alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSȘȚUVWXYZ

Encryption Table (key = 3)

x   |  0|  1|  2|  3|  4|  5|  6|  7|  8|  9| 10| 11| 12| 13| 14| 15| 16| 17| 18| 19| 20| 21| 22| 23| 24| 25| 26| 27| 28| 29| 30|
m   |  C|  R|  I|  P|  T|  O|  G|  A|  F|  E|  Ă|  Â|  B|  D|  H|  Î|  J|  K|  L|  M|  N|  Q|  S|  Ș|  Ț|  U|  V|  W|  X|  Y|  Z|
x+k |  3|  4|  5|  6|  7|  8|  9| 10| 11| 12| 13| 14| 15| 16| 17| 18| 19| 20| 21| 22| 23| 24| 25| 26| 27| 28| 29| 30|  0|  1|  2|
c   |  P|  T|  O|  G|  A|  F|  E|  Ă|  Â|  B|  D|  H|  Î|  J|  K|  L|  M|  N|  Q|  S|  Ș|  Ț|  U|  V|  W|  X|  Y|  Z|  C|  R|  I|

Input:  CIFRULCEZAR
Result: POÂTXQPBIĂT

=== Romanian Caesar Cipher ===
1. Standard Caesar Cipher
2. Two-Key Caesar Cipher
0. Exit
Choose cipher:
```

Figure 3.3 - Task 1.2: encryption and the displayed permuted alphabet

Comparison. With the same numeric key and prepared input, the standard version gives `FKITXOFHÂBT`, while the permuted version gives `POÂTXQPBIĂT`. The difference comes entirely from the keyword-derived alphabet order.

3.2.1 Recovering the two-key plaintext

To decrypt, the user selects mode 2 and operation 2. The numeric key is 3 and the keyword is `criptografie`. The ciphertext is `POÂTXQPBIĂT`. The program rebuilds the same displayed permutation and returns `CIFRULCEZAR`.

Romanian Caesar Cipher C# program · live standard input/output

```
=== Romanian Caesar Cipher ===
1. Standard Caesar Cipher
2. Two-Key Caesar Cipher
0. Exit
Choose cipher: 2

1. Encrypt
2. Decrypt
Choose operation: 2
Enter numeric key (1-30): 3
Enter keyword (at least 7 letters): criptografie
Enter message: POÂTXQPBIĂT

Keyword: CRIPTOGRAFIE
Original alphabet: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ
Permuted alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSȘȚUVWXYZ

Encryption Table (key = 3)

x   |  0|  1|  2|  3|  4|  5|  6|  7|  8|  9| 10| 11| 12| 13| 14| 15| 16| 17| 18| 19| 20| 21| 22| 23| 24| 25| 26| 27| 28| 29| 30|
m   |  C|  R|  I|  P|  T|  O|  G|  A|  F|  E|  Ă|  Â|  B|  D|  H|  Î|  J|  K|  L|  M|  N|  Q|  S|  Ș|  Ț|  U|  V|  W|  X|  Y|  Z|
x+k |  3|  4|  5|  6|  7|  8|  9| 10| 11| 12| 13| 14| 15| 16| 17| 18| 19| 20| 21| 22| 23| 24| 25| 26| 27| 28| 29| 30|  0|  1|  2|
c   |  P|  T|  O|  G|  A|  F|  E|  Ă|  Â|  B|  D|  H|  Î|  J|  K|  L|  M|  N|  Q|  S|  Ș|  Ț|  U|  V|  W|  X|  Y|  Z|  C|  R|  I|

Input:  POÂTXQPBIĂT
Result: CIFRULCEZAR

=== Romanian Caesar Cipher ===
1. Standard Caesar Cipher
2. Two-Key Caesar Cipher
0. Exit
Choose cipher:
```

Figure 3.4 - Task 1.2: decryption using the same numeric key and keyword

Manual check. In the permuted alphabet, P has position 3 and decrypts to position 0, containing C. O has position 5 and decrypts to position 2, containing I. I in the ciphertext has position 2 and decrypts to position 30, containing Z. These checks confirm that decryption also uses positions in the permuted alphabet.

3.3 Invalid numeric keys and message characters

Figure 3.5 shows three rejected numeric keys: `0`, `31` and `abc`. Each attempt produces `Invalid key. Enter an integer between 1 and 30.`, followed by a new key prompt. Once the valid key `3` is supplied, the program continues to the message.

The messages `salut!` and `salut2` are rejected. Each error names the offending character, displays the accepted alphabet and states that spaces are allowed. The message prompt is repeated after each error. A subsequent valid input `Salut` was accepted and encrypted to `ȚBOXV`, confirming that the program recovers from invalid input.



Figure 3.5 - Rejected keys and message characters, followed by repeated prompts

3.4 Invalid keywords

Figure 3.6 shows that the two-key mode also rejects the numeric key `-1`. The key `30` is accepted. The six-letter keyword `secret` produces the minimum-length error; `cripto1` is rejected because it contains a digit, and `cripto grafie` is rejected because keywords cannot contain spaces. Each error is followed by a new keyword prompt.

Romanian Caesar Cipher C# program · live standard input/output

```
=== Romanian Caesar Cipher ===
1. Standard Caesar Cipher
2. Two-Key Caesar Cipher
0. Exit
Choose cipher: 2

1. Encrypt
2. Decrypt
Choose operation: 1
Enter numeric key (1-30): -1
Invalid key. Enter an integer between 1 and 30.
Enter numeric key (1-30): 30
Enter keyword (at least 7 letters): secret
Invalid keyword: Keyword must contain at least 7 letters.
Enter keyword (at least 7 letters): cripto1
Invalid keyword: Character '1' is not allowed. Allowed letters: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ.
Enter keyword (at least 7 letters): cripto grafie
Invalid keyword: Character ' ' is not allowed. Allowed letters: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ.
Enter keyword (at least 7 letters):
```

Figure 3.6 - Two-key validation: wrong shift, short keyword, digit and space

3.5 Romanian letters and boundary checks

With the standard cipher, $k = 1$ and input `aăâîşţz`, normalization produces `AĂÂÎȘȚZ`; encryption produces `ĂÂBJTUA`. Figure 3.7 demonstrates lowercase conversion, cedilla compatibility, Romanian diacritics and the wraparound $Z \to A$.



Figure 3.7 - Romanian letters, cedilla normalization and wraparound at key 1

The two-key mode was also checked with $k_1 = 30$, the seven-letter keyword `ştiinţă` and the same input. The normalized keyword `ȘTIINȚĂ` yields `ȘTINȚĂAÂBCDEFGHÎJKLMOPQRSUVWXYZ`; the result is `ĂȚAHZNY`. This checks the upper key boundary and shows that a valid seven-letter keyword can include a repeated letter.

Additional checks exercised every numeric key from 1 to 30 over all 31 letters: once for the standard cipher and once for each of the keywords `criptografie` and `ştiinţă`. All 90 encryption/decryption round trips passed. Each encryption was also compared with an independently calculated modular mapping. In total, these checks executed the unchanged program 180 times. Console transcripts and the verification summary are saved in the report's `evidence` folder.

4 Conclusions

In this laboratory work, I implemented and examined two Caesar ciphers for the Romanian alphabet: a fixed-alphabet version and a version with a keyword-based permutation. I used explicit positions from 0 to 30, rather than character encoding values, and applied modular arithmetic to encrypt and decrypt the text. The results confirmed that decryption with the same key or keys recovers the prepared plaintext.

The shared normalization and validation functions handle Romanian diacritics, cedilla variants, lowercase letters and spaces in messages. The console retry loops reject invalid keys, unsupported characters and keywords that are too short. The displayed permutation and mapping tables make both algorithms easy to verify by hand. These checks helped me understand why alphabet order and consistent preprocessing are essential for correct encryption and decryption.

The standard version is weak because an attacker needs to try only 30 numeric keys. A keyword permutation greatly increases the theoretical parameter space, but the second version still uses one fixed substitution for the entire message and preserves letter frequencies and repeated patterns. Therefore, neither cipher is suitable for modern secure communication. The main value of these implementations is learning substitution ciphers, modular arithmetic and reliable input handling.

4.1 Source code

The full source code is available in the GitHub repository [2]:

`https://github.com/MiHCraft/Cryptography_UTM`

The implementation for this laboratory work is `lab_01/code/Program.cs`. The report describes and quotes this file; no changes were made to the implementation while preparing the report.

Bibliography

[1] Technical University of Moldova. *Laboratory Work No. 1: Caesar Cipher*. Course assignment supplied as `Laboratory Work no. 1. Caesar Cipher.pdf`, pp. 1–4.

[2] Pancenco, Mihail. *Cryptography_UTM*. Romanian Caesar Cipher implementation. `lab_01/code/Program.cs`. `https://github.com/MiHCraft/Cryptography_UTM`.